



Rapport Sécurité & Améliorations

Livraison Rapide OUAGA & BOBO

Date : 9 juin 2026

Auteur : Claude Sonnet 4.6 (Anthropic)

Projet : Plateforme de livraison — Burkina Faso

Stack : Next.js · Supabase · TypeScript · PWA

CONFIDENTIEL — USAGE INTERNE

Rapport Complet — Sécurité & Améliorations

Livraison Rapide OUAGA et BOBO

Date : 2026-06-09 Auteur : Claude Sonnet 4.6 (Anthropic)

1. Corrections de Sécurité Appliquées

C-1 — RLS basé sur `user_metadata` (CRITIQUE)

Fichiers modifiés : `fixsecurityv2.sql` (nouveau), `hooks/useSupabaseAuth.ts`

Problème : Toutes les politiques RLS (Row Level Security) Supabase utilisaient `auth.jwt()->'usermetadata'->'role'` pour vérifier si un utilisateur est administrateur. Or, `user_metadata` est modifiable par n'importe quel utilisateur via `supabase.auth.updateUser()`. N'importe qui pouvait donc s'auto-promouvoir admin.

Correction :

- `fixsecurityv2.sql` (Étape 1) : Mise à jour de `rawappmetadata` avec `{"role":"admin"}` pour le compte admin (via `UPDATE auth.users SET raw appmetadata = ... WHERE phone LIKE '%67370909%'`)
- `fixsecurityv2.sql` (Étape 2) : Réécriture de TOUTES les politiques RLS pour utiliser uniquement `(auth.jwt()->'app_metadata'->'role') = 'admin'`
- `fixsecurityv2.sql` (Étape 3) : Mise à jour de la vue `livreursview` pour n'utiliser que `ctx.app role` (suppression de la condition sur `user_metadata->'phone'`)
- `hooks/useSupabaseAuth.ts` : Suppression du fallback `session.user.usermetadata?.role` et de la vérification du numéro de téléphone en dur. Seul `app metadata?.role` est désormais utilisé.

Tables protégées : `livreurs`, `clientslivraison`, `deblocages`, `chats livraison`, `tickets_support`, `annonces`, `paiements`

Impact : Aucun impact sur le compte admin. Le compte admin conserve tous ses accès car son `app_metadata.role` est maintenant correctement défini côté serveur.

C-2 — Endpoint `/api/push` ouvert sans authentification (HAUTE)

Fichier modifié : `app/api/push/route.ts`

Problème : L'endpoint POST `/api/push` acceptait des requêtes non authentifiées. Tout acteur malveillant pouvait envoyer une notification push à n'importe quel utilisateur de la plateforme en passant simplement un `recipientId`.

Correction : Ajout d'une vérification de session en début de handler via `createServerClient` SSR :

```
const cookieStore = await cookies();
const supabaseServer = createServerClient(cookieStore);
const { data: { session } } = await supabaseServer.auth.getSession();
if (!session) return NextResponse.json({ error: 'Unauthorized' }, { status: 401 });
```

Impact : Seuls les utilisateurs authentifiés (livreurs et clients connectés) peuvent désormais déclencher des notifications push vers un autre utilisateur. Le flux de messagerie interne est préservé.

C-3 — `/api/verify-payment` sans authentification (NON MODIFIÉ - INTENTIONNEL)

Fichier : `app/api/verify-payment/route.ts`

Ce comportement est intentionnel : le flux de paiement Orange Money est anonyme par conception. L'endpoint fait appel à l'IA Gemini pour valider les reçus sans nécessiter de compte. **Aucune modification effectuée** conformément à la demande.

H-4 — Headers de sécurité HTTP manquants (HAUTE)

Fichier modifié : `next.config.mjs`

Problème : Absence totale de headers de sécurité HTTP sur toutes les réponses de l'application.

Headers ajoutés (sur toutes les routes `/(.*)`) :

Header	Valeur	Protection
X-Frame-Options	SAMEORIGIN	Clickjacking
X-Content-Type-Options	nosniff	MIME-sniffing
Referrer-Policy	strict-origin-when-cross-origin	Fuite d'URL
X-DNS-Prefetch-Control	on	Performance + sécurité DNS
Permissions-Policy	geolocation=(self), camera=(self), microphone=()	Abus de permissions navigateur

Impact : Aucun impact fonctionnel. Les headers de service worker conservent leur comportement de cache.

H-5 — `image_url` des chats non validée (HAUTE)

Fichier modifié : `components/ChatDrawer/ChatDrawer.tsx`

Problème : L'URL d'image dans les messages chat (`msg.image_url`) était rendue directement dans un `<img src=...>` sans vérification de l'origine. Un attaquant pouvait stocker une URL arbitraire dans la base de données et déclencher des requêtes vers des serveurs tiers (tracking, phishing visuel).

Correction : Ajout d'une fonction `isSafeMediaUrl()` qui vérifie que le hostname de l'URL correspond au domaine Supabase configuré :

```
const isSafeMediaUrl = (url: string): boolean => {
  try {
    const { hostname } = new URL(url);
    return hostname === SUPABASE_HOSTNAME || hostname.endsWith('.supabase.co');
  } catch { return false; }
};
```

Les images dont l'URL n'est pas issue de Supabase Storage sont simplement ignorées à l'affichage.

Impact : Seules les images téléversées via Supabase Storage (flow normal) sont affichées. Aucun impact sur l'expérience utilisateur normale.

M-1 — Paywall sessionStorage côté client (NON MODIFIÉ - INTENTIONNEL)

Fichier : Composant de vérification de paiement

Ce mécanisme est intentionnel pour le flux d'accès temporaire. **Aucune modification effectuée** conformément à la demande.

2. Améliorations du Compte Admin

2a. Nouvelles capacités d'action (AdminDashboard.tsx)

Capacité	Description
Activer/Désactiver Premium client	Bouton toggle dans l'onglet Clients pour modifier <code>subscription_paid</code> directement
Supprimer un client	Suppression définitive avec confirmation dans l'onglet Clients
Désactiver une annonce	Bouton "Désactiver" sur chaque annonce active dans les Paramètres
Retirer badge vérifié	Bouton pour retirer <code>is_verified</code> sur un livreur actif
Réactiver un livreur suspendu	Même bouton "Valider" si statut = suspendu
Export CSV Livreurs	Téléchargement de tous les livreurs au format CSV
Export CSV Clients	Téléchargement de tous les clients au format CSV
Export CSV Paiements	Téléchargement de tous les paiements vérifiés au format CSV
Broadcast Push	Envoi d'une notification push à tous les appareils enregistrés

2b. Nouvelles données visibles

Donnée	Source
Revenus réels confirmés (FCFA)	Somme des paiements avec <code>statut = 'VALIDE'</code>
Livreurs Ouagadougou vs Bobo	Filtrage par <code>city = 'ouaga'</code> / <code>city = 'bobo'</code>
Visiteurs estimés	Calcul basé sur clients + vues de profils
Vues profil & clics contact total	Somme des <code>viewscount</code> et <code>contacts count</code>
Dossiers incomplets vs prêts à valider	Filtrage des documents manquants
Annonces actives listées	Affichage avec bouton de désactivation rapide

2c. Recherche / Filtrage

- Recherche livreurs** : Filtre en temps réel par nom / prénom / téléphone

- **Recherche clients** : Filtre en temps réel par nom / téléphone

3. Analyses Journalières (Nouvel Onglet)

Fichier modifié : `hooks/useAdminStats.ts`

Nouvelles interfaces :

```
export interface DailyStat {
  date: string;          // "2026-06-01"
  label: string;         // "01 juin"
  newDrivers: number;    // nouveaux livreurs inscrits ce jour
  newClients: number;    // nouveaux clients inscrits ce jour
  messages: number;     // messages échangés ce jour
  payments: number;     // paiements validés ce jour
}
```

Données calculées : Sur les **14 derniers jours** glissants, en parallèle via `Promise.all`.

Visualisation dans le Dashboard

Onglet "Analytiques" comprend :

1. **4 cartes KPI 14j** — Total sur 14 jours : Livreurs, Clients, Messages, Paiements
2. **Bar chart CSS natif** — Graphique à barres groupées (4 métriques par jour) avec légende colorée
3. **Répartition géographique** — Ouagadougou vs Bobo avec barre de progression et pourcentage

4. API Admin Broadcast (Nouveau Fichier)

Fichier créé : `app/api/admin/broadcast/route.ts`

Fonctionnement :

- Vérifie que la session est active (401 sinon)
- Vérifie que `session.user.app_metadata.role === 'admin'` (403 sinon)
- Récupère TOUTES les `push_subscriptions` via le service role Supabase
- Envoie la notification à chaque appareil enregistré
- Nettoie automatiquement les abonnements expirés (HTTP 404/410)
- Retourne `{ success: true, sent: N }` avec le nombre d'envois réussis

Sécurité : Double protection — session requise + rôle admin vérifié côté serveur via `app_metadata`.

5. Script SQL à Appliquer

Fichier : `fixsecurityv2.sql`

⚠ ORDRE D'APPLICATION CRITIQUE : Ce script doit être exécuté dans Supabase SQL Editor **AVANT** tout déploiement du nouveau code. Dans le cas contraire, le compte admin perdrait temporairement ses accès.

Étapes du script :

- 1. Mise à jour de `rawappmeta_data` avec `{"role":"admin"}` pour le compte admin
- 2. Recréation de toutes les politiques RLS en utilisant uniquement `app_metadata`
- 3. Mise à jour de la vue `livreurs_view`
- 4. Instructions pour les buckets Supabase Storage (actions manuelles)

6. Résumé des Fichiers Modifiés / Créés

Fichier	Action	Description
<code>fixsecurityv2.sql</code>	Créé	Migration RLS vers app_metadata
<code>hooks/useAdminStats.ts</code>	Réécrit	Ajout DailyStat, ouagaDrivers, boboDrivers, totalRevenuePaid, toggleClientPremium, deleteClient, deactivateAnnonce
<code>components/AdminDashboard/AdminDashboard.tsx</code>	Réécrit	Onglet analytics, recherche, toggle premium, broadcast, CSV, city stats
<code>hooks/useSupabaseAuth.ts</code>	Modifié	Suppression fallback user_metadata et vérification phone hardcodée
<code>app/api/push/route.ts</code>	Modifié	Ajout vérification auth session (C-2)
<code>app/api/admin/broadcast/route.ts</code>	Créé	Endpoint broadcast push admin-only
<code>next.config.mjs</code>	Modifié	Ajout headers de sécurité HTTP (H-4)
<code>components/ChatDrawer/ChatDrawer.tsx</code>	Modifié	Validation domaine Supabase pour image_url (H-5)

7. Actions Manuelles Restantes (Supabase Dashboard)

Ces actions ne peuvent pas être effectuées via SQL et nécessitent le Dashboard Supabase :

1. **Bucket** `chatimages` → *Storage* → *chatimages* → Politiques → Modifier la politique INSERT pour exiger `TO authenticated` (retirer `TO public`)
 2. **Bucket** `recus-paiements` → *Storage* → *recus-paiements* → Edit bucket → Décocher "Public bucket" (les reçus ne seront plus accessibles sans URL signée)
-

Rapport généré automatiquement le 2026-06-09

Rapport généré automatiquement le 9 juin 2026 · Livraison Rapide OUAGA & BOBO